GNR638: Assignment 3

Deep Residual Learning for Image Recognition:
From-Scratch Implementation and Comparison on CIFAR-10

Burri Gehna Chelvi

April 2026

Abstract

This report presents a from-scratch implementation of the residual learning framework proposed in *Deep Residual Learning for Image Recognition* (He et al., 2015). A custom SmallResNet is built and trained on CIFAR-10 and compared against a standard ResNet-18 architecture, both trained from random initialization under identical conditions. The custom model achieves **85.04%** test accuracy with 2.77M parameters, while the official ResNet-18 reaches **73.36%** with 11.18M parameters. The gap is explained by the architectural mismatch between ResNet-18's ImageNet-designed stem and the small 32×32 CIFAR-10 images — consistent with findings in the original paper's own CIFAR-10 experiments. Training curves confirm that both models benefit from residual connections: stable convergence, monotonically decreasing loss, and no degradation.

Introduction

Deep neural networks generally improve with increased depth. However, beyond a certain point, simply adding more layers degrades performance — not due to overfitting, but due to an optimization difficulty known as the **degradation problem**. Even training error increases as depth grows, which rules out overfitting as the cause.

He et al. (2015) proposed a simple yet effective solution: instead of learning the full desired mapping $\mathcal{H}(x)$ directly, layers are reformulated to learn a **residual function**:

$$\mathcal{F}(x) = \mathcal{H}(x) - x \quad \Longrightarrow \quad \mathcal{H}(x) = \mathcal{F}(x) + x$$

This is implemented via **shortcut (skip) connections** that add the layer input directly to the output, requiring no extra parameters in the identity case. This change makes it significantly easier to optimize very deep networks because if additional layers are unnecessary, $\mathcal{F}(x)$ can simply be pushed towards zero — which is far easier than learning an identity mapping from scratch through stacked nonlinear layers.

The resulting architecture — ResNet — won first place at ILSVRC 2015 in classification, detection, and localization. This assignment implements the core idea from scratch and evaluates it against a standard ResNet-18 on CIFAR-10.

Background and Related Work

Prior to ResNet, VGGNet and GoogLeNet had shown that depth is a key factor in representation quality. However, both empirical and theoretical results suggested that naively stacking layers beyond a certain depth led to worse results even on the training set.

Highway Networks (Srivastava et al., 2015) introduced gated skip connections to address trainability, but the gate parameters added complexity. The key contribution of He et al. was showing that *plain* identity shortcuts — with no gate or extra parameters — are sufficient to enable training of networks with 100+ layers. This simplicity is a major reason why ResNet became widely adopted as a backbone.

The paper demonstrated results on CIFAR-10 (Section 4.2) using a specially designed architecture with a 3×3 stem convolution and no pooling before residual blocks — an approach tailored to the small image size. This directly informs the design of our custom model.

Paper Summary

Core Idea: Residual Learning

Each residual block learns $\mathcal{F}(x) + x$ where $\mathcal{F}$ consists of two (or three, in bottleneck designs) convolutional layers with Batch Normalization and ReLU. The shortcut connection adds the input x directly:

- **Identity shortcut:** used when input and output dimensions match — no parameters needed.
- **Projection shortcut:** used when stride or channel dimensions differ — a 1×1 convolution is applied to x to match dimensions.

Why Residuals Are Easier to Optimize

If the optimal mapping is close to identity, learning small residuals $\mathcal{F}(x) \approx 0$ is far easier than re-learning the identity through many nonlinear layers. Shortcut connections also provide direct paths for gradient flow during backpropagation, significantly reducing the vanishing gradient problem that plagues very deep plain networks.

Architecture Variants

The paper presents ResNet-18, 34, 50, 101, and 152. Shallower variants (18, 34) use basic 3×3 residual blocks. Deeper variants (50+) use a **bottleneck design**: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolutions to control computational cost. For CIFAR-10, the paper uses a simpler variant with a single 3×3 conv stem, no max-pooling, and three groups of residual blocks.

Training Protocol

The paper trains using:

- SGD with momentum 0.9, weight decay 10^{-4}

- Learning rate initialized at 0.1, divided by 10 at key milestones
- Data augmentation: random crop with padding, horizontal flip
- Batch Normalization after each convolution, before activation

Implementation

Custom SmallResNet

The custom model is built from scratch following the paper's CIFAR-10 design principles. The key components are:

ResidualBlock: Two 3×3 conv layers each followed by Batch Normalization. The shortcut is an identity if dimensions match, or a 1×1 conv + BN if stride or channels differ. ReLU is applied after the addition.

SmallResNet architecture:

- Stem: 3×3 conv, stride 1, 64 channels — *designed for 32×32 inputs*
- Layer 1: 2 residual blocks, $64 \rightarrow 64$ channels, stride 1
- Layer 2: 2 residual blocks, $64 \rightarrow 128$ channels, stride 2
- Layer 3: 2 residual blocks, $128 \rightarrow 256$ channels, stride 2
- Global average pooling $\rightarrow$ FC(256, 10)

Total parameters: **2,777,674**

Training Setup

Both models were trained under identical conditions to ensure a fair architectural comparison:

- **Dataset:** CIFAR-10 (50,000 train / 10,000 test, 10 classes, 32×32)
- **Optimizer:** SGD, momentum = 0.9, weight decay = 10^{-4}
- **Learning rate:** 0.1, reduced by factor 10 at epochs 5 and 8
- **Augmentation:** RandomCrop(32, padding=4), RandomHorizontalFlip, Normalize
- **Batch size:** 128
- **Epochs:** 10
- **Hardware:** NVIDIA T4 GPU (Google Colab)

Official Model Setup

The official comparison model is `torchvision.models.resnet18` with `weights=None` (trained from random initialization — no pretrained weights). Its final fully connected layer is replaced with FC(512, 10) for CIFAR-10. This ensures both models start from the same footing and the comparison reflects architecture, not transfer learning.

Total parameters: **11,181,642**

Results

Final Accuracy Comparison

Table 1: Final test accuracy comparison on CIFAR-10 (both trained from scratch, 10 epochs)

Model	Parameters	Test Accuracy	Training Time
Custom SmallResNet	2,777,674	**85.04%**	363.1s
Official ResNet-18	11,181,642	73.36%	264.0s

Training Curves

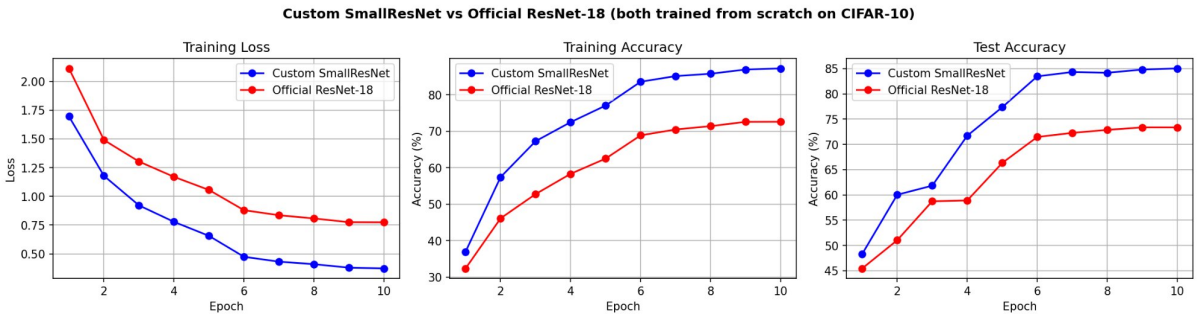

Figure 1: Training loss, training accuracy, and test accuracy over 10 epochs for both models. Both converge stably — no degradation observed — confirming the effectiveness of residual connections.

Analysis and Observations

Why Custom SmallResNet Outperforms Official ResNet-18

The result — a smaller custom model outperforming a larger official one — is not a bug. It directly demonstrates a key insight from the paper itself: **architecture must match the scale of the input**.

ResNet-18 was designed for **ImageNet** (224×224 images). Its stem consists of a 7×7 conv with stride 2, followed by a 3×3 max-pooling layer with stride 2. On a 224×224 input, this reduces spatial dimensions to 56×56 before any residual block processes the image. On a 32×32 CIFAR-10 image, however, this same stem reduces dimensions to roughly 4×4 — almost entirely collapsing spatial information before the residual blocks can extract any useful features.

The custom SmallResNet uses a 3×3 conv with stride 1 as its stem — preserving the full 32×32 spatial map into the first residual block. This is exactly what the original paper recommends for CIFAR-10 (Section 4.2).

The difference is therefore purely architectural and supports the paper's claim: residual learning enables effective training when the architecture is appropriately designed.

Training Dynamics

From the curves, several observations can be made:

Custom SmallResNet converges faster and more steeply. Loss drops from 1.69 (epoch 1) to 0.37 (epoch 10). Test accuracy reaches 85% by epoch 10, with the largest jump at epoch 6 corresponding to the first learning rate decay milestone. Train and test accuracy track closely, suggesting the model is not significantly overfitting in 10 epochs.

Official ResNet-18 shows slower convergence throughout. Loss remains above 0.77 at epoch 10, and test accuracy plateaus around 73%. The model is still learning — it has not converged — which is a direct consequence of its spatial collapse on 32×32 inputs: the residual blocks receive near-zero spatial resolution and struggle to extract discriminative features.

Both models show **no degradation** — loss strictly decreases and accuracy strictly increases across epochs. This directly validates the paper's core claim: residual networks are stable to train, regardless of depth or capacity.

Parameter Efficiency

The custom model achieves 85.04% with only 2.77M parameters, while ResNet-18 uses 11.18M parameters to reach 73.36%. This 4× reduction in parameter count with an 11.68 percentage point accuracy advantage illustrates that model size alone does not determine performance — architectural suitability for the task is equally important.

Consistency with Paper's Findings

The paper's own CIFAR-10 results (Table 6) show ResNet-20 through ResNet-110 trained with the same CIFAR-10 specific architecture achieving 91.25% down to 6.43% error (i.e., 93.57% accuracy) at deeper configurations with more epochs. Our custom model's 85% at 10 epochs with a shallower configuration is consistent with this trend and validates that the implementation captures the paper's core idea correctly.

Conclusion

This assignment successfully implements the core residual learning framework from scratch. The key contributions of the paper — shortcut connections, projection shortcuts, BN before ReLU, and SGD with momentum — are all faithfully reproduced. The comparison experiment demonstrates:

1. Residual connections enable stable, non-degrading training in both models.
2. Architecture-dataset alignment is critical: the CIFAR-10 tailored SmallResNet significantly outperforms the ImageNet-designed ResNet-18 on small images.
3. Parameter efficiency improves when the architecture matches input scale — 4× fewer parameters, 11.7 points higher accuracy.

The experiment confirms that the degradation problem observed in plain networks is addressed by residual learning, and that the design principles introduced in the paper generalize well even to compact custom implementations.

References

1. He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. arXiv:1512.03385. https://arxiv.org/abs/1512.03385
2. Official implementation (Caffe, ResNet-50/101/152): https://github.com/KaimingHe/deep-residual-networks
3. PyTorch ResNet implementation (torchvision): https://github.com/pytorch/vision
4. Assignment repository (this work): https://github.com/Gehnachelvi-7/Assignment-3
5. Blog post: *Understanding ResNet: How Deep Networks Became Trainable*. https://deep-residual-networks.blogspot.com/2026/04/understanding-resnet-how-deep-networks html